Conversational Intake Service

Telegram-to-webapp capture for projects · Product Brief & Alignment Document

INTERNAL DRAFT · V1

This document describes a proposed service that lets workshop staff feed information — captured during phone calls, site visits, or messaging — directly into active projects through a Telegram bot, with an LLM resolving which project the input belongs to and pre-filling structured fields for human confirmation. It states the purpose, the assumptions the design rests on, the data and service expectations the application places on the existing web application, and the open questions that must be aligned before build. A dedicated section tracks the difference between what the design currently *assumes* the sidecar web application API provides and what it **actually exposes in production today**.

HOW TO READ THIS DOCUMENT

The brief is written so the gap section (§7) can be revised independently as the production API is inspected. Everywhere the design depends on an API capability, that dependency is named and cross-referenced, so when production reality differs, only the affected rows need updating — not the whole document.

1 Purpose & what the service tries to achieve

Today, information gathered away from a desk — a client phone call, a supplier quote, a measurement taken on the shop floor — is either memorised, written on paper, or typed into the web application later, if at all. The delay and the manual re-entry are where detail is lost.

The service closes that gap. A staff member sends whatever they have — a voice note, a typed message, photos, or any combination — to a Telegram bot the moment they have it. The service transcribes and reads the input, infers which active project it concerns, extracts the relevant field values, and presents them back for a quick item-by-item confirmation before writing to the project.

Capture at source

Information enters the system when and where it happens, not hours later from memory.

Zero-form entry

Staff speak or type naturally. The service does the structuring, not the human.

Confirmed, not blind

Nothing is written until a human validates each field. Speed without silent errors.

Primary success measure

The service succeeds if it is **faster and lower-friction than the current path** (open laptop → find project → type) for the median capture event, while keeping data quality at or above manual entry. If staff find it slower or less trustworthy than typing, it has failed regardless of how clever the inference is.

2 How it works — end to end

The pipeline below is the canonical flow. Section 7 maps each stage that touches the web application to a specific API dependency.

Capture (text · audio · images) › **Normalise** (transcribe · read) › **Infer project** (LLM + context) › **Resolve** (auto · clarify · ask) › **Confirm** (field by field) › **Apply** (write to project)

Project resolution — the three branches

Confidence	Behaviour	Rationale
High · 1 match ≥ 0.75	Auto-select, show a confirm badge, go straight to field review.	Interrupting a confident match wastes the user's time. Trust, then verify at the field stage.
Partial · 2–3 matches	Show 2–3 candidates as inline buttons; best guess pre-highlighted.	One tap resolves ambiguity. Cheaper than forcing a search.
Low · 0 matches	Fall back to project search / full active list.	Better to ask than to write to the wrong project.

MANDATORY-OPTIONAL INPUT RULE

Either **text or audio must be present** in every submission — images alone cannot anchor project inference reliably, because they rarely contain the project name or context the LLM needs. Images are enrichment, not a standalone trigger. This is a design constraint, not a limitation to be removed later without re-validating inference quality.

3 Assumptions the design rests on

Each assumption below is something the design takes as true. If one proves false in your environment, the linked section needs revisiting before build.

#	Assumption	If false…
A1	Active projects have stable, machine-readable identifiers the bot can reference.	Resolution and write-back both break — see §4, §7.
A2	Each project has a known set of fields with types the LLM can target.	Field extraction degrades to free-text notes only.
A3	Staff using the bot are authorised to write to the projects they reference.	An identity/permission model must be added before launch — see Q6.
A4	The set of active projects is small enough that LLM context can hold meaningful matching hints.	A pre-filter/search step is needed ahead of the LLM call.
A5	Most captures concern a project that already exists in the system.	A "create project from capture" path is needed (currently out of scope).
A6	Staff are comfortable with European Portuguese voice and informal phrasing.	Transcription and prompt language settings need revisiting.
A7	A short confirmation step is acceptable on every capture.	A "trusted auto-apply above X% confidence" mode must be designed (raises data-quality risk).

4 Data expectations of the web application

For the service to perform as presented, the web application must be able to provide and accept the following. These are stated as *capabilities required*, deliberately independent of how the current API happens to expose them — §7 reconciles the two.

Read side — what the bot needs to fetch

Capability	Shape expected by design	Used for
List active projects	`id` , `name` , `status` , `description`	Candidate set for resolution
Project field schema	`key` , `label` , `type` , `options[]` , current value	Targeted extraction & field review
Match hints design-added	`aliases[]` , `key_contacts[]` , `typical_materials[]` , `priority_fields[]`	Raises inference accuracy & orders the confirm card
Existing field data	current value per field	To compare extracted vs. existing before writing

Write side — what the bot needs to submit

Capability	Shape expected by design	Used for
Submit an entry	`project_id`, `fields{}`, `source`, `submitted_by`, `timestamp`, `raw_transcript`	Applying confirmed data to the project
Attach media design-added	image/audio references linked to the entry	Audit trail & later human review
Idempotency / dedupe	a client-supplied key per capture	Prevent double-writes on retries

WHY "COMPARE BEFORE WRITE" MATTERS

The design extracts a value, then compares it to what the project already holds, and surfaces the difference in the confirmation card. This is only possible if the read side returns **current field values**, not just the schema. If production returns schema-without-values, the confirm card can still work but loses the "this changes X from Y to Z" framing — a meaningful UX downgrade tracked in §7.

5 Service & infrastructure expectations

Transcription

A speech-to-text capability handling PT-PT robustly under phone-call audio quality. Self-hosted (Whisper) or API. Latency under a few seconds per note is the comfort threshold.

LLM inference

A model that reliably returns strict JSON against a provided schema, reasons over PT-PT, and fits the active-project context. Local (Ollama) or API. Must degrade gracefully when uncertain.

Bot runtime & session state

A long-running bot process holding per-conversation state through the multi-step confirm flow. Needs durable-enough storage to survive a restart mid-capture.

Network reachability

The bot runtime must reach both the web application API and the inference services. Outbound to Telegram required; inbound webhook or long-poll either works.

COST & PRIVACY POSTURE

Running transcription and inference on local infrastructure keeps capture data inside the workshop network and removes per-call API cost — a natural fit if existing local inference is already in place. The trade-off is hardware headroom and maintenance. This is an alignment decision, not a fixed design choice (see Q4).

6 Open questions to align

These must be answered before or during build. Each is tagged by the area it most affects. Unanswered, they become assumptions — and assumptions in §3 are where risk concentrates.

Q1 **What is the canonical identifier and "active" definition for a project in production?** DATA MODEL
Drives resolution and write-back. Must match exactly what the API returns and accepts.

Q2 **Does every project type share a field schema, or do schemas vary per project?** DATA MODEL
Determines whether the bot fetches one schema or per-project schemas, and how generic the confirm card must be.

Q3 **Who curates the match-hints (aliases, contacts, typical materials) and where do they live?** INFERENCE
Inference accuracy depends heavily on these. They are net-new data the current model may not hold.

Q4 **Local vs. cloud for transcription and LLM — what is the cost, privacy, and latency preference?** INFRA
Shapes infrastructure, recurring cost, and where capture data physically resides.

Q5 **How does a Telegram user map to a web-application identity, and what may they write?** SECURITY
No permission model is assumed yet (A3). Required before any production write.

Q6 **Is creating a new project from a capture in scope, or strictly out?** SCOPE
A5 assumes the project exists. If captures can precede project creation, a new path is needed.

Q7 **Should high-confidence captures ever auto-apply without field confirmation?** UX POLICY

A7 assumes always-confirm. An auto-apply mode trades speed for data-quality risk.

Q8 **What is the expected volume of captures per day, and the active-project count?** SIZING

Validates A4 (context fits) and sizes infrastructure and pre-filtering needs.

7 Planned API vs. production reality — reconciliation

This is the section to revise as the production API is inspected. The left side states what the design currently **expects**; the right side is to be filled in with what production **actually exposes today**. Status reflects the gap.

Matches production already provides this · Partial exists but shape/behaviour differs · Missing not available yet · TBC not yet inspected

Capability	Design expects	Production has today	Status
List active projects	`GET /projects?status=active` → id, name, status, description	*— to confirm —*	TBC
Project field schema	typed fields with options and labels	*— to confirm —*	TBC
Current field values	values returned alongside schema	*— to confirm —*	TBC
Match hints	aliases, contacts, materials, priority fields	*likely not present — net new*	Missing
Submit entry	`POST /entries` with source + transcript	*— to confirm —*	TBC
Media attachment	link image/audio to entry	*— to confirm —*	TBC
Idempotency key	client key to dedupe retries	*— to confirm —*	TBC
Identity / auth	Telegram user → app identity + scope	*— to confirm —*	TBC

ADAPTATION STRATEGY FOR THE GAP

Where production lacks a capability, there are three moves, in order of preference: **(1)** extend the web application API to provide it cleanly; **(2)** add a thin adapter layer in the bot runtime that synthesises it from what production does expose (e.g. deriving match-hints from a separately curated file); **(3)** degrade the feature gracefully and note the UX cost (e.g. drop "changes X to Y" framing if current values aren't available). Each row above should land on one of these three once production is inspected.

8 Out of scope for v1

- Creating new projects from a capture (pending Q6).
- Editing or deleting existing entries through the bot — capture is additive only.
- Auto-apply without confirmation (pending Q7).
- Multi-project single-capture splitting — one capture targets one project.
- Languages beyond PT-PT and technical English.